

總整理：

DOM

```
// getElement 系列
document.getElementById()
document.getElementsByClassName()

// querySelector 系列
document.querySelector()
document.querySelectorAll()
```

Handwritten annotations in the image:

- A yellow bracket under `getElementById()` points to the handwritten text `"Id"`.
- A pink bracket under `getElementsByClassName()` points to the handwritten text `"class"`.
- A yellow bracket under `querySelector()` points to the handwritten text `"#Id"`.
- A pink bracket under `querySelectorAll()` points to the handwritten text `".class"`.

題外話：

第一種抓法印出的是HTMLCollection

第二種抓法印出的是NodeList

NodeList可以做更多事

```
// 取得 class 是 doll 的元素們 (全部)
let allDolls1 = document.getElementsByClassName("doll")
let allDolls2 = document.querySelectorAll(".doll")

console.log(allDolls1) // HTMLCollection
console.log(allDolls2) // NodeList
```

DOM更多操作：
createElement()
textContent
appendChild()

```
const hello = document.querySelector("#hello")

// 建立一個 H1 元素
const h = document.createElement("h1")
h.textContent = "學習 JavaScript"

// 再另外建立一個 div 元素
const d = document.createElement("div")
d.textContent = "hi"

// 把元素加到 hello 裡，並放在最後面
hello.appendChild(h)
hello.appendChild(d)
```

1. `innerHTML` (會解析 HTML)

它會把你提供的字串當成 **HTML 原始碼** 來處理。如果你字串裡有標籤（如 `<b>`），它會渲染出效果。

- **優點**：可以用來插入新的標籤結構。
- **缺點**：效能較差（因為瀏覽器要重新解析 HTML），且有 **跨站腳本攻擊 (XSS)** 的安全風險。

2. `textContent` (只看純文字)

它會把你提供的內容全部當成 **純文字**。即使裡面有 HTML 標籤，也會直接被當成普通文字顯示出來。

- **優點**：效能好、**安全**（不會執行惡意腳本）。
- **特性**：會讀取到所有文字內容，包含被 CSS 隱藏的文字。

DOM更多操作：
remove()
removeChild()

// 用 remove() / removeChild() 拿掉ul的最後一個li

```
const btn = document.querySelector("#removeBtn");
```

```
btn.addEventListener("click", () => {  
  const lastOne = document.querySelector("li:last-child");
```

```
  if (lastOne) {  
    const u = document.querySelector("ul");  
    u.removeChild(lastOne);
```

*// 另一個寫法是：拿掉上面這兩行，改成
lastOne.remove();這行即可。*

```
  }  
});
```

DOM更多操作：
parentElement
children

// 用 parentElement() 取得上層元素

```
const lastOne = document.querySelector("li:last-child")
```

取得上層 Element

```
console.log(lastOne.parentElement)
```

取得上層 Node

```
console.log(lastOne.parentNode)
```

// 建議用 parentElement

// 用 children 取得下層元素

```
const ul = document.querySelector("ul")
```

取得子層 Element

```
console.log(ul.children)
```

取得子層 Node

```
console.log(ul.childNodes)
```

// 建議用 children

DOM更多操作：
previousElementSibling
nextElementSibling

// 用 *previousElementSibling* / *nextElementSibling* 取得兄弟姐妹元素

```
const li = document.querySelector("li:nth-child(2)");
```

// *Element* 系列

取得上一個;

```
console.log(li.previousElementSibling);
```

取得下一個;

```
console.log(li.nextElementSibling);
```

// *Node* 系列

取得上一個;

```
console.log(li.previousSibling);
```

取得下一個;

```
console.log(li.nextSibling);
```

DOM更多操作：
insertAdjacentElement("",)
insertAdjacentHTML("",)

// 用insertAdjacentElement("位置", 變數或常數名稱) 在指定位置安插元素物件

```
const ul = document.querySelector("ul");
```

```
const li = document.createElement("li");  
li.textContent = "X";
```

```
ul.insertAdjacentElement("afterend", li);
```

// 用insertAdjacentHTML("位置", 變數或常數名稱) 在指定位置安插HTML字串

```
const ul = document.querySelector("ul");
```

```
const li = "<li>Z</li>";
```

```
ul.insertAdjacentHTML("afterbegin", li);
```

// 建議用insertAdjacentHTML

beforebegin

afterbegin

 悟空

 克林

 達爾

beforeend

afterend

抓之後 加上監聽器
addEventListener

寫法二把函數單獨列出來
如果有其他按鈕要做類似效果
就可以共用這個函數 比較方便

寫法一

把fn當引數
回呼函數
Callback function

寫法二

```
// 取得 DOM
const btn = document.querySelector("#btn")

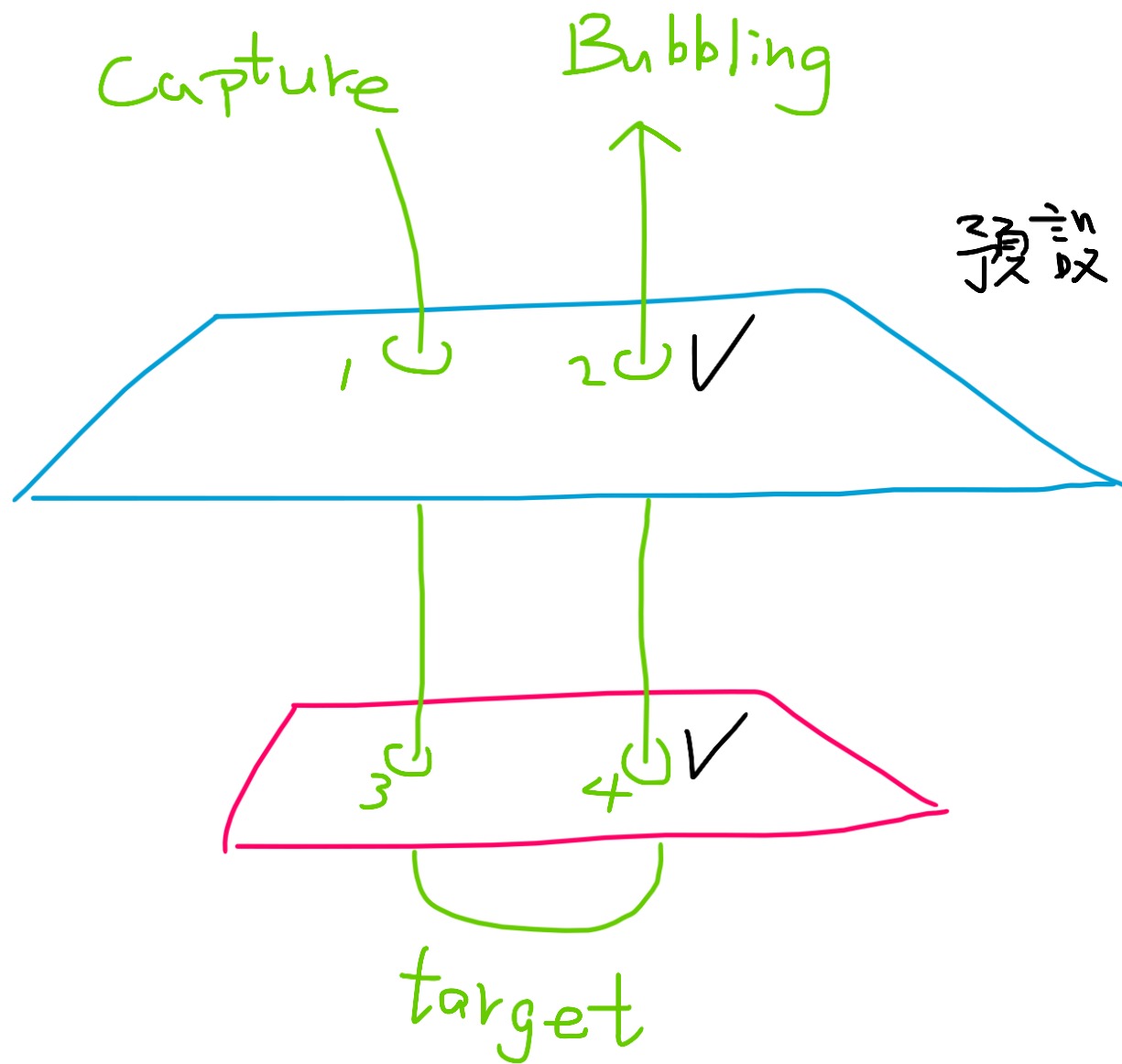
// 監聽 click 事件，當發生的時候就...
btn.addEventListener("click", function () {
  console.log("我被按了")
})
```

```
// 取得 DOM
const btn = document.querySelector("#btn")

const clickHandler = function () {
  console.log("我被按了")
}

// 把函數當做引數傳給它
btn.addEventListener("click", clickHandler)
```

Event Flow



如果把連JS這行寫在head可能會導致抓不到想抓的東西
(因為還沒讀取完)

解決方法：把連JS這行寫在body最後
但有時候我們必須把連JS這行寫在head.....

解決辦法：在JS寫一個「對整個文件document的監聽器」
瀏覽器頁面載入完成後 會發生DOMContentLoaded事件
此時監聽器監聽到「完成載入了」
再去執行抓的任務(querySelector)

```
14  
15   <script src="app.js"></script>  
16 </body>
```

連js

```
index.html  JS  app.js  ×  
JS app.js > document.addEventListener("DOMContentLoaded") callback  
1  document.addEventListener("DOMContentLoaded", () => {  
2    const btn = document.querySelector("#btn")  
3  
4    console.log(btn)  
5  })  
6
```

```
<script defer src="413.js"></script>
```

更快的解決辦法：
在script後面加上defer(延遲運作)
就不用去js寫document.addEventListener("DOMContentLoaded", () => {
});

檔案總管 ... 0413.html x JS 0413.js

✓ JAVASCRIPT

- > .vscode
- JS 0402.js
- JS 0407.js
- 0413.html
- JS 0413.js

0413.html > html > body > button#btn

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>Document</title>
7   <!-- <script src="0413.js"></script> -->
8   <script defer src="0413.js"></script>
9 </head>
10 <body>
11   <button id="btn">123</button>
12 </body>
13 </html>
```

檔案總管 ... 0413.html JS 0413.js x

✓ JAVASCRIPT JS 0413.js > ...

- > .vscode
- JS 0402.js
- JS 0407.js
- 0413.html
- JS 0413.js

```
1 // document.addEventListener("DOMContentLoaded", () => {
2 //   const nums = document.querySelector("#btn");
3 //   nums.addEventListener("click", () => {
4 //     console.log(123);
5 //   });
6 // });
7 const nums = document.querySelector("#btn");
8 nums.addEventListener("click", () => {
9   btn.textContent = "456";
10 });
11
```

```
檔案總管  ...  0413.html X  0413.js
JAVASCRIPT
> .vscode
  0402.js
  0407.js
  0413.html
  0413.js

1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Document</title>
7     <!-- <script src="0413.js"></script> -->
8     <script defer src="0413.js"></script>
9   </head>
10  <body>
11    <!-- <button id="btn">123</button> -->
12
13    <!-- <button id="Minus">-</button>
14    <input type="text" id="nums" value="1" />
15    <button id="Plus">+</button> -->
16
17    身高
18    <input type="text" id="height" />
19    <br />
20    體重
21    <input type="text" id="weight" />
22    <br />
23    結果
24    <input type="text" id="bmi" />
25    <button id="btn">計算</button>
26  </body>
27 </html>
28
```

身高	170	
體重	70	
結果	24.22	計算

```
0413.js > ...
30 // minus.addEventListener("click", () => {
31 //   nums.value = nums.value - 1;
32 //   if (nums.value - 1 < 0) {
33 //     nums.value = 0;
34 //   }
35 // });
36
37 const btn = document.querySelector("#btn");
38 const bmi = document.querySelector("#bmi");
39 const height = document.querySelector("#height");
40 const weight = document.querySelector("#weight");
41
42 btn.addEventListener("click", () => {
43   const h = Number(height.value) / 100;
44   const w = Number(weight.value);
45   // bmi.value = w / h ** 2;
46   const result = w / h ** 2;
47   bmi.value = result.toFixed(2);
48 });
49
```

onclick寫法
和addEventListener監聽器有一樣的效果

在同一顆按鈕重複執行
只會印出最新那次
(前面的被覆蓋了)

```
// 取得 DOM
const btn = document.querySelector("#btn")

// 設定當被 click 的時候要做什麼事
btn.onclick = function () {
  console.log("我被按了")
}
```

```
btn.onclick = function () {
  console.log("1")
}

btn.onclick = function () {
  console.log("2")
}
```

```
btn.addEventListener("click", () => {
  console.log("1")
})
btn.addEventListener("click", () => {
  console.log("2")
})
})
```

但addEventListener可以重複很多次不被覆蓋

```
// 取得 DOM
const link = document.querySelector("#link")

// 監聽 click 事件，當發生的時候就...
link.addEventListener("click", function (e) {
  e.preventDefault() // 把原本預設的行為停下來
  console.log("我被按了")
})
```

預設讓它不要執行轉去超連結的動作
只要去印「被按了」

練習題

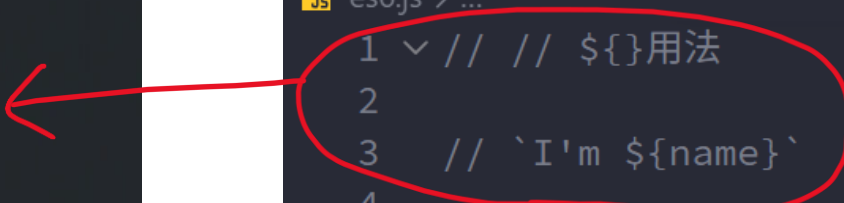
ES6

```
let name = "悟空"
let age = 18

// 用 + 號串接
console.log("大家好，我的名字是" + name + "，我今年" + age + "歲")

// 把變數帶進去
console.log(`大家好，我的名字是${name}，我今年${age}歲`)
```

```
JS es6.js > ...
1  ✓ // // ${}用法
2
3  // `I'm ${name}`
4
5  const name = hero.name
6  const age = hero.age
7  這兩行可以寫成這樣：
8  const {name, age} = hero
9  console.log(name, age)
```



物件{}解構

```
✓ const cat = {  
✓ name: "kk",  
✓ age: 18,  
}
```

const {name, age} = cat
name: name 的哪一個 name 代表上面的 "kk"?

✓ 其實長這樣 ✓
{name: name, age: age}
new new

右邊的 `name` 代表物件裡面的屬性值（也就是 "kk"）。

你可以這樣拆解記憶：

{ 來源屬性名 : 新變數名稱 } = 物件

1. 冒號左邊的 `name`：指的是 `cat` 物件裡的 Key（屬性名稱）。程式會去 `cat` 裡面找叫作 `name` 的東西，發現它的值是 "kk"。
2. 冒號右邊的 `name`：是你新宣告的變數名稱。它接收了前面找到的 "kk"。

concat()寫法

```
index.html  JS app.js  X  ...
JS app.js > ...
1  const H1 = ["A", "B", "C"]
2  const H2 = ["X", "Y", "Z"]
3  const HA11 = H1.concat(H2)
4
5  console.log(HA11)
6
```

主控台 >> 1 1

篩選 預設層級 ▾

1 個問題: 1

app.js:5
(6) ['A', 'B', 'C', 'X', 'Y', 'Z']

...H1,...H2寫法

```
index.html  JS app.js  X  ...
JS app.js > [e] HA11
1  const H1 = ["A", "B", "C"]
2  const H2 = ["X", "Y", "Z"]
3  // const HA11 = H1.concat(H2)
4  const HA11 = [...H1, ...H2]
5
6  console.log(HA11)
```

主控台 >> 1 1

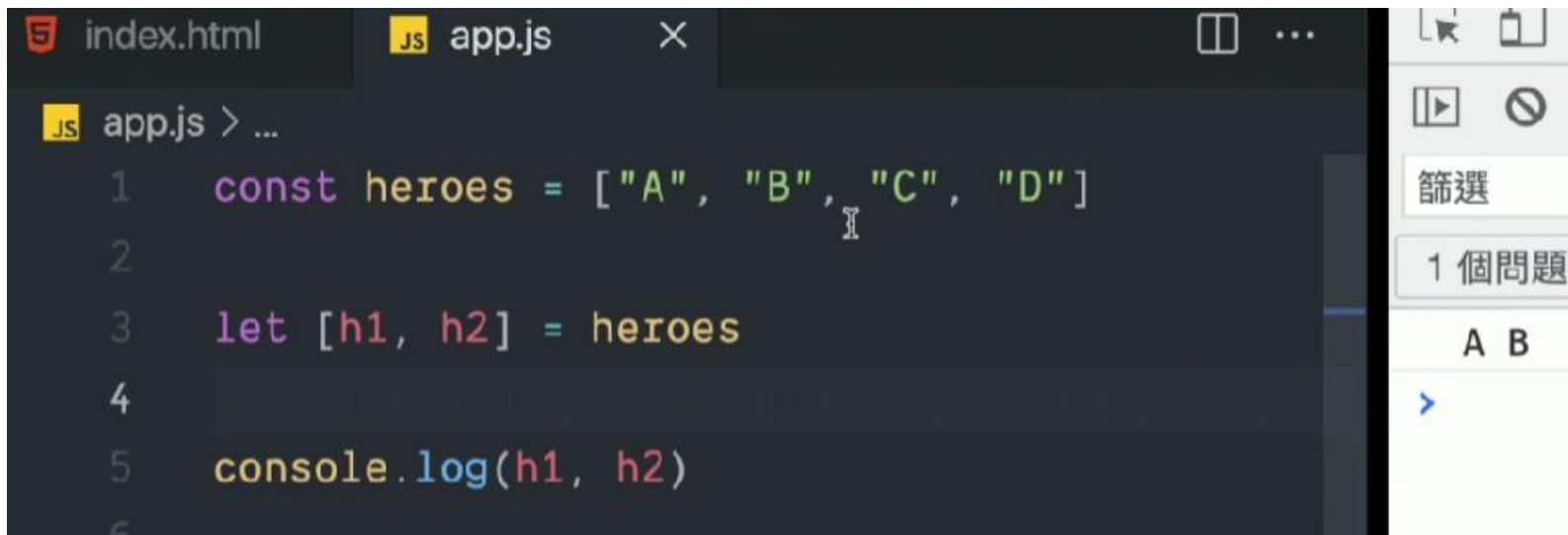
篩選 預設層級 ▾

1 個問題: 1

app.js:6
(6) ['A', 'B', 'C', 'X', 'Y', 'Z']

「...」其餘運算子

陣列[]解構



The image shows a code editor with two tabs: 'index.html' and 'app.js'. The 'app.js' tab is active, displaying the following JavaScript code:

```
1  const heroes = ["A", "B", "C", "D"]
2
3  let [h1, h2] = heroes
4
5  console.log(h1, h2)
```

The code demonstrates array destructuring. A constant array 'heroes' is defined with the values 'A', 'B', 'C', and 'D'. Then, the first two elements of the array are assigned to variables 'h1' and 'h2' using the destructuring syntax. Finally, 'console.log' is used to output the values of 'h1' and 'h2'.

On the right side of the editor, there is a sidebar with a search bar labeled '篩選', a section titled '1 個問題', and a list of items labeled 'A B' with a blue arrow pointing to the right.

「...」其餘運算子

```
11 // // 「...」收展功能(它的作用是把「剩餘的所有參數」都收集起來，  
    並自動包裝成一個陣列)。它只能擺在最後面 表示剩下的我c全部都要 若  
    擺在其他位置會導致語法錯誤。  
12 function hi(a, b, ...c) {  
13   console.log(a, b, c);  
14 }  
15 hi("x", "y", "z", "k");  
16
```

問題 輸出 偵錯主控台 終端機 連接埠

```
PS C:\Users\Galaxy\Desktop\JavaScript> node es6.js  
x y [ 'z', 'k' ]  
PS C:\Users\Galaxy\Desktop\JavaScript>
```

```
17 function myLog(a, b, ...c) {  
18   console.log(a, b, ...c);  
19 }  
20 myLog(1, "a", 2, 3, "5", 7);  
21
```

問題 輸出 偵錯主控台 終端機 連接埠

```
PS C:\Users\Galaxy\Desktop\JavaScript> node es6.js  
1 a 2 3 5 7  
PS C:\Users\Galaxy\Desktop\JavaScript>
```

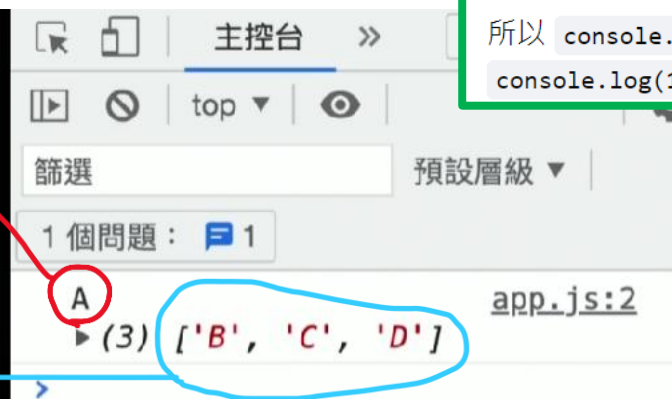
你在印出時寫的是 `console.log(a, b, ...c)` :

1. `a` 是 `1`
2. `b` 是 `"a"`
3. `...c` : 這三個點會把 `c` 陣列裡的內容「拆解、攤平」出來。
 1. 原本 `c` 是 `[2, 3, "5", 7]`
 2. `...c` 展開後變成 `2, 3, "5", 7`

所以 `console.log` 接收到的其實是 :

```
console.log(1, "a", 2, 3, "5", 7);
```

```
index.html app.js  
app.js > sayHello  
1 function sayHello(u1, ...u2) {  
2   console.log(u1, u2)  
3 }  
4  
5 sayHello("A", "B", "C", "D")  
6
```



```

22 function addOne(...a) {
23     return a.map((i) => i + 1);
24 }
25 const result = addOne(1, 3, 5, 8, 10, 0, -1);
26 console.log(result); // [2, 4, 6, 9, 11, 1, 0]
27

```

問題 輸出 偵錯主控台 終端機 連接埠

+ v powershell

PS C:\Users\Galaxy\Desktop\JavaScript> node es6.js

```

[
  2, 4, 6, 9,
  11, 1, 0
]

```

PS C:\Users\Galaxy\Desktop\JavaScript>

```

28 function addOne(...a) {
29     return a.filter((i) => typeof i == "number").map((i)
    => i + 1);
30 }
31 const result = addOne(1, 3, 5, 8, "a", 10, 0, -1,
    false);
32 console.log(result); // [2, 4, 6, 9, 11, 1, 0]
33

```

問題 輸出 偵錯主控台 終端機 連接埠

+ v powershell

PS C:\Users\Galaxy\Desktop\JavaScript> node es6.js

```

[
  2, 4, 6, 9,
  11, 1, 0
]

```

PS C:\Users\Galaxy\Desktop\JavaScript>

整包nums被帶入a位置
b和c沒東西 → undefined

```
36 const nums = [1, 2, 3, 4, 5];
37 function hi(a, b, c) {
38   console.log(a, b, c);
39 }
40 hi(nums);
41
```

問題 輸出 偵錯主控台 終端機 連接埠

```
PS C:\Users\Galaxy\Desktop\JavaScript> node es6.js
[ 1, 2, 3, 4, 5 ] undefined undefined
PS C:\Users\Galaxy\Desktop\JavaScript>
```

...nums意思是把nums展開(不再包成一個陣列)

```
36 const nums = [1, 2, 3, 4, 5];
37 function hi(a, b, c) {
38   console.log(a, b, c);
39 }
40 hi(...nums);
41
```

問題 輸出 偵錯主控台 終端機 連接埠

```
PS C:\Users\Galaxy\Desktop\JavaScript> node es6.js
1 2 3
PS C:\Users\Galaxy\Desktop\JavaScript>
```

```
135 //陣列的 .toString() 非常執著於把內容「打平」成一個清單。如果內容是數字或字串，就直接顯示；如果內容是
    //另一個陣列，就拆掉中括號繼續打平；只有遇到「真正的物件 {}」時，才會祭出 [object Object] 這個大絕招。
136 //物件{}轉字串會是'[object Object]'
137 //[]+[] = "" + "" = ""
138 //[]+{} = "" + "[object Object]" = "[object Object]"
139 //{ }+[] = block + "" = block + 0 = 0
140 // 因為{}放前面會變成block 也就是空的沒東西 然後加號放在東西前面會讓那個東西變數字 也就是+ "" = 0
141 //{ }+{} = block + {} = NaN
142 // 因為{}放前面會變成block 也就是空的沒東西 然後加號放在東西前面會讓那個東西變數字 也就是+ {} = NaN
143 //簡單來說空物件不能強行轉數字(+ {} = NaN)但是空字串可以強行轉數字(+ "" = + 0 = 0)
```

XMLHttpRequest()

抓取網路資料

```
const api = "https://jsonplaceholder.typicode.com/posts"

const req = new XMLHttpRequest()
req.addEventListener("load", function () {
  const posts = JSON.parse(req.responseText)
  // 處理 posts 的顯示
})
req.open("GET", api)

req.send()
```

前面要記得加new

fetch

抓取網路資料

```
const api = "https://jsonplaceholder.typicode.com/posts"

fetch(api)
  .then(function (response) {
    return response.json()
  })
  .then(function (data) {
    // 處理 posts 的顯示
  })
```

json功能：讀取並解析資料，它會告訴瀏覽器：「請幫我把抓到的原始文字，轉換成 JavaScript 看得懂的物件/陣列」。

功能 	querySelector	fetch
主要目的	選取 DOM 元素 (HTML 標籤)	發送 網路請求 (HTTP Request)
執行對象	當前頁面已有的 HTML 結構	遠端伺服器或 API 接口
結果類型	回傳一個 Element 物件 (如按鈕、文字)	回傳一個 Promise 物件 (包含資料)
同步/非同步	同步 (立刻找到並選取)	非同步 (需等待網路傳輸)

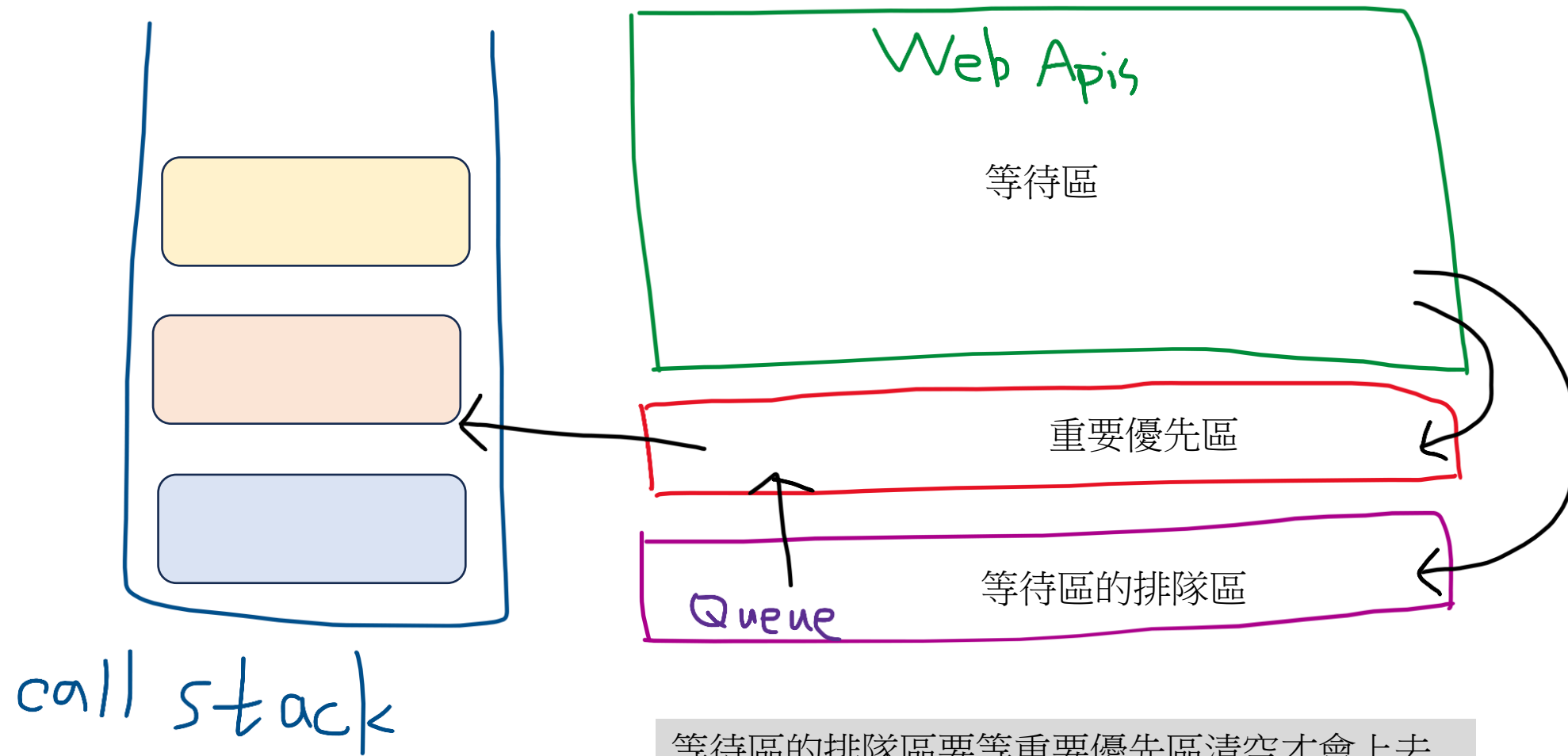
async和await 抓取網路資料

```
const api = "https://jsonplaceholder.typicode.com/posts"

async function getPost() {
  const response = await fetch(api)
  const posts = await response.json()

  // 處理 posts 的顯示
}

getPost()
```



等待區的排隊區要等重要優先區清空才會上去
重要優先區要等call stack區清空才會上去

<video></video> 影片

controls = 可以讓使用者控制播放、音量等
autoplay = 會自動播放，若有音訊的瀏覽器會停用自動播放
loop = 影片播完自動重播

```
<video src="/cooking.mp4">瀏覽器不支援 video 標籤時會顯示的文字</video>
```

查詢 HTML 標籤以及 CSS 樣式的支援度

70

<source></source>

媒體或影像來源元素
type = 指定媒體類型
source 標籤用於指定 video 標籤支持的多個視頻來源。
提供不同格式或質量的影片，讓瀏覽器選擇最適合的來播放。

```
<video controls>
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  <source src="video.ogv" type="video/ogg">
  瀏覽器不支援 video 標籤時會顯示的文字
</video>
```

71

<audio></audio> 音訊元素

音樂播放器、Podcast
controls = 可以讓使用者控制播放、音量等
autoplay = 會自動播放，若有音訊的瀏覽器會停用自動播放
loop = 影片播完自動重播

```
<audio controls>
  <source src="myAudio.mp3" type="audio/mpeg" />
  <source src="myAudio.ogg" type="audio/ogg" />
  瀏覽器不支援 audio 標籤時會顯示的文字
</audio>
```

72

<track></track> 影片字幕

作為媒體元素的子元素
需要有字幕 .vtt 檔，需要加上 default 屬性

```
<video controls src="/cooking.mp4">
  <track kind="subtitles" src="/cooking.vtt" srclang="zh"
    label="中文" />
  瀏覽器不支援 video 標籤時會顯示的文字
</video>
```

73

`<iframe></iframe>` 內嵌框架

允許你在當前頁面內嵌入另一個 HTML 頁面
例如 Google 地圖、YouTube 影片、Twitter 論壇



<details></details> 詳細資訊

可展開和折疊的內容區塊，使用者可以點擊來查看或隱藏內容
會搭配 <summary> 使用，經常用在 FAQ 頁面，open 屬性可打開

常見問題

Q1: Cake 如何幫助我找到工作？



在 Cake 國際人才社群平台，您可以一站式透過如[職缺搜尋功能](#)一覽海內外最新工作機會，進入[履歷編輯器](#)、[作品集編輯器](#)快速建立專業履歷和作品集，運用 [Meet 功能](#)拓展專業人脈網絡，並搭配求職[內容資源](#)、[主題專區](#)輔助，幫助您找到理想且合適的工作。Cake 上的雇主也能透過人才搜尋引擎、Cake 的 AI 人才配對等方式找到並主動聯繫您。

Q2: 雇主、獵才顧問、或其他人如何在 Cake 上找到我？



<summary></summary>

折疊區塊的標題，告知使用者點擊可以查看更多內容。

FAQ

常見問題

Q1: Cake 如何幫助我找到工作？



在 Cake 國際人才社群平台，您可以一站式透過如[職缺搜尋功能](#)一覽海內外最新工作機會，進入[履歷編輯器](#)、[作品集編輯器](#)快速建立專業履歷和作品集，運用 [Meet 功能](#)拓展專業人脈網絡，並搭配求職[內容資源](#)、[主題專區](#)輔助，幫助您找到理想且合適的工作。Cake 上的雇主也能透過人才搜尋引擎、Cake 的 AI 人才配對等方式找到並主動聯繫您。

Q2: 雇主、獵才顧問、或其他人如何在 Cake 上找到我？



<svg></svg> 標記元素

向量圖（文字檔）增加 SEO、檔案小、可以做動畫、Logo、有透明度。可以直接放在程式碼上，也可以使用 img。

基本元素

rect
circle
line
path
polygon

矩形
圓形
直線
用於複雜圖形
多邊形

常用屬性

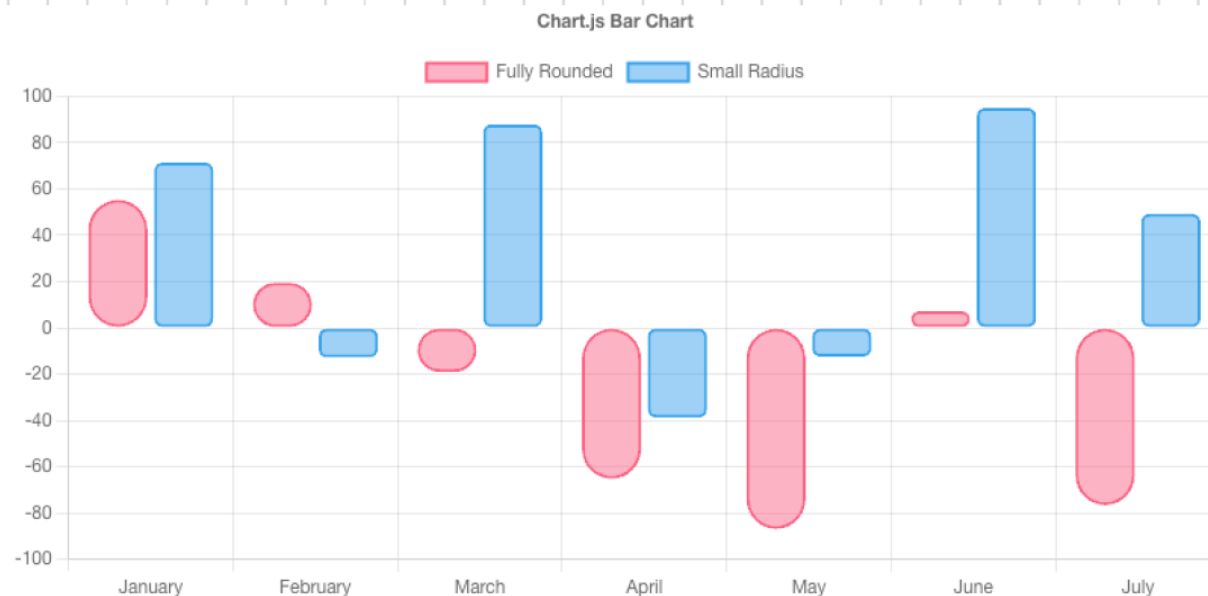
stroke
stroke-width
fill

邊框色
邊框寬度
圖形顏色

`<canvas></canvas>` 畫布

用來繪製圖形、顯示動態內容、圖像處理、互動應用。
本身是一個容器，沒有可見內容。要用 JavaScript 繪製內容。

Chart.js



`<script></script>` 腳本

用來在 HTML 文件中嵌入或引用 JavaScript 代碼，主要功能是使網頁具有動態和互動性。它可以放在頁面的 `<head>` 或 `<body>`

```
<script src="../../js/table.js" defer></script>
```

css 單位介紹 - 絕對單位

單位	意思	說明
✓ px	pixel 像素	1px 等於 1/96 英吋，反之 1 英吋內有 96 個像素，就是螢幕上非常小的單位（最常用）
pt	point 點	1px 等於 1/72 英吋，文書軟體常見單位
pc	pica 皮卡	1px 等於 1/6 英吋，印刷單位

css 單位介紹 - 相對單位 1

單位	意思	說明
em	父層字體大小	假如父層字體大小為 20px，那在子層設定 2em 就會是 40px，會受父層影響。
✓ rem	根元素的字體大小	假如根元素字體大小為 16px，那在子層設定 2rem 就會是 32px，統一受根元素影響。
ex	小寫 x 的高度	根據當前字體中小寫字母 x 的高度來計算。
ch	character width 字符寬度	以數字 0 在當前字體中的寬度來設定元素的尺寸或間距。

css 單位介紹 - 相對單位 2

單位	意思	說明
✓ %	百分比	相對父層寬高百分比，假如父層寬度為 100px，那 50% 就會是 50px。
✓ vw	視口(螢幕)寬度	80vw 等於 8/10 視口寬度；100vw 等於100% 視口寬度
✓ vh	視口(螢幕)高度	同上，寬度換成高度
vmax	視口(螢幕)較大值	寬高中較大的那一邊，例如: 寬 1024px；高 768px，10vmax 是指 1024 的 10%。
vmin	視口(螢幕)較大值	vmax 的相反
fr	分數 Fraction	用在 grid 中，一個 fr 是一個隔線空間

css 寬. 高 - width. height

屬性	意思	說明
width	寬	width: auto; fit-content(跟內容一樣) inline 類型物件不支援 width 與 height
height	高	height: 50%; inline 類型物件不支援 width 與 height
min-width/ max-width	最小寬/ 最大寬	min-width: 200px; 不會小於 200px 通常在寬高沒有寫絕對單位的情況下使用 互相衝突時 min>max>一般
min-height/ max-height	最小高/ 最大高	

css 內外距 - padding. margin

屬性	意思	說明
padding	內距	padding: 上 右 下 左; 寫4個 padding: 上 左右 下; 寫3個 padding: 上下 左右; 寫2個 padding: 上下左右; 寫1個 百分比是依據父層的「寬」作為 100%
margin	外距	margin: 30% auto; (水平居中) 百分比是依據父層的內容空間計算

css 盒模型 - box-sizing

屬性	意思	說明
box-sizing	元素的寬高計算方式	box-sizing: content-box; (預設) border-box



CSS 群組宣告

```
<ul class="news-list">
  <li class="news-list-item">1</li>
  <li class="news-list-item">2</li>
  <li class="news-list-item">3</li>
</ul>
<ul class="info-list"></ul>
  <li class="info-list-item">a</li>
  <li class="info-list-item">b</li>
  <li class="info-list-item">c</li>
</ul>
```

```
.news-list {
  color: red;
}
```

```
.info-list {
  color: red;
}
```

記得加逗號

```
.news-list, .info-list {
  color: red;
}
```


css 組合式宣告

```
<a href="#" class="btn" id="link-btn">按鈕樣式</a>  
<button type="button" class="btn" id="just-btn">按鈕樣式  
</button>
```

a#link-btn.btn
button.btn#just-btn
(不需要空格)



CSS 層疊式宣告

```
<ul class="news-list">
  <li class="news-list-item">1</li>
  <li class="news-list-item">2</li>
  <li class="news-list-item">3</li>
</ul>
<ul class="info-list"></ul>
  <li class="info-list-item">a</li>
  <li class="info-list-item">b</li>
  <li class="info-list-item">c</li>
</ul>
```

```
.news-list-item {
  color: red;
}
```

```
.info-list-item {
  color: red;
}
```

記得加逗號

```
.news-list li, .info-list li {
  color: red;
}
```

css 親代選取器 +

```
<h1>標題</h1>  
<p>第一段</p>  
<p>第二段</p>  
<p>第三段</p>
```

h1 + p (只有第一個 p)

如果想選到除了第一個p後面更多p
就用h1~p(在下一頁)

```
<ul class="list">  
  <li class="list-item">a</li>  
  <li class="list-item">b</li>  
  <li class="list-item">c</li>  
  <li class="list-item">d</li>  
  <li class="list-item">e</li>  
  <li class="list-item">f</li>  
</ul>
```

li + li (排除第一個 li)

css 親代選取器 ~

這兩邊示範是一樣的意思
反正h1後的p都有被選到

```
<h1>標題</h1>  
<p>第一段</p>  
<p>第二段</p>  
<p>第三段</p>
```

h1 ~ p (全部的 p)

```
<h1>標題</h1>  
<p>第一段</p>  
<p>第二段</p>  
<p>第三段</p>  
<h2>標題2</h2>  
<p>第四段</p>  
<h3>標題3</h3>  
<p>第五段</p>
```

h1 ~ p (一到五段 p 都有)

css 子代選取器 >

```
<div class="first-item">  
  <div class="second-item">  
    <div class="third-item"></div>  
  </div>  
</div>
```

.first-item > div (只會選到 second-item)

有 > 與沒有 > 的差異

計算優先權

CSS 優先權

<div> . <p> . input . btn

!important > inline style > ID >
class/屬性選取器 > Tag > * > 繼承

繼承就是從父層繼承來的
比如color、font-size

```
.danger {  
  background-color: red !important;  
}
```

*div *{ } *{ } 這2個都可用*

!important -> 加在樣式後面

class -> .class名稱

屬性選取器 -> img[alt] 、 img[alt="可愛貓咪圖"]、

img[alt^="可愛"] (開頭等於)、img[src\$=".jpg"] (結尾等於)

img[alt*="可愛"] (包含文字)、img[alt~="fly"] (包含英文單字)

需注意大小寫. 算分

屬性選取器

常用於input

/* 只要有寫 alt 屬性的圖片，就加一個灰色外框 */

```
img[alt] {  
  border: 2px solid #ccc;  
}
```

/* 只有 alt 剛好等於 "可愛貓咪圖" 的那張圖會變圓角 */

```
img[alt="可愛貓咪圖"] {  
  border-radius: 50%;  
}
```

/* 只要 alt 是以 "可愛" 開頭的（例如：可愛小狗、可愛風景），都套用樣式 */

```
img[alt^="可愛"] {  
  box-shadow: 5px 5px 10px gold;  
}
```

/* 只要圖片路徑結尾是 .jpg 的，加一個紅色邊框 */

```
img[src$=".jpg"] {  
  border: 3px solid red;  
}
```

/* 只要 alt 裡面有出現 "可愛" 兩個字（例如：這隻貓超可愛、可愛的鳥），通通放大 */

```
img[alt*="可愛"] {  
  transform: scale(1.1);  
}
```

/*

HTML 若為 alt="super fly boy" -> 會中

HTML 若為 alt="butterfly" -> 不會中（因為 fly 不是獨立單字）

*/

```
img[alt~="fly"] {  
  opacity: 0.8;  
}
```